SoK: Reshaping Research on Network Intrusion Detection Systems

Giovanni Apruzzese
University of Liechtenstein
Vaduz, Liechtenstein
Reykjavik University
Reykjavik, Iceland
giovannia@ru.is

Abstract

Network Intrusion Detection Systems (NIDS) have been studied for decades. Hundreds of papers have, e.g., proposed ways to enhance, harden or bypass NIDS. However, the findings of prior literature are hardly reflected in real-world operational contexts. Such a disconnection is problematic for research itself: it is unclear what scenario envisioned by prior work can be used as a baseline for future advancements.

We argue that a key reason for this disconnection is a fundamental misunderstanding of intrinsic characteristics of NIDS. For instance, the fact that a compromised NIDS cannot be expected to work well; the fact that some evaluations are done without carrying out any experiment in a (even synthetic) "real" network; the fact that security operators triage high-level reports—and not individual samples flagged by some classifier. In this SoK, which is primarily a reflective piece, we first constructively highlight such quintessential properties (without criticizing *any* work by different authors) by stating three Assertions. Then, we provide recommendations—further emphasized through an original and reproducible case study that challenges some established practices. Ultimately, we seek to lay a foundation to reshape research on NIDS.

CCS Concepts

• **Computing methodologies** → **Machine learning**; • **Security and privacy** → **Network security**; **Intrusion detection systems**.

Keywords

machine learning, attack detection, false positives, threat model

ACM Reference Format:
Giovanni Apruzzese. 2026. SoK: Reshaping Research on Network Intrusion Detection Systems. In *ACM Asia Conference on Computer and Communications Security (ASIA CCS '26), June 1–5, 2026, Bangalore, India.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3779208.3807488

1 Introduction

Research on Network Intrusion Detection Systems (NIDS) is constantly increasing (see Fig. 1). Since 2015, thousands of articles

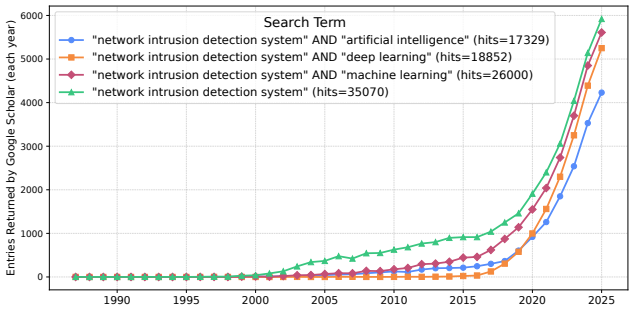

Fig. 1: Research interest in NIDS. Queries issued on March 2026.

appear yearly on NIDS, especially in conjunction with machine-learning (ML) techniques. Such articles propose new methods that improve NIDS-related components [18], or extend NIDS functionalities [135], or bypass existing NIDS [30]; some papers also systematized [23] or "troubleshooted" [62] our knowledge on this domain.

We acknowledge the contributions of prior work. However, we believe that, over time, scientific literature on NIDS has been focusing excessively on the "research" aspect—losing sight of the real-world implications of NIDS-related technologies. As a potential consequence, a recent survey among industry practitioners revealed skepticism towards the results claimed in research [23].

We want to change this. This "systematization of knowledge" (SoK) paper attempts to trigger a reflective exercise in the reader. We do so by stating three ASSERTIONS, rooted in established security principles, and which we believe should not be forgotten by future work aiming to advance the state of the art in NIDS.

1.1 Goal and Methodology (and disclaimer)

This SoK paper is rooted on the following assumption: *research in NIDS is relatively stagnant, and is hindered by, among others, a superficial treatment of the NIDS domain.* By "superficial treatment," we intend: *(a)* overlooking essential security principles; or *(b)* ignoring the characteristics of modern networks; or *(c)* neglecting the basic function of NIDS—or any combination of the previous points.[1]

Hence, our goal is to provide a constructive foundation that can put an end to this stagnation. Specifically, our overarching objective is to devise a concise *vademecum*—a guiding reference—that can help (re)shape future research on NIDS. In doing so, we adopt a positive stance toward prior literature. Indeed, we will treat past works by following three rules:

[1]The assumptions/ASSERTIONS of this paper are not only supported by this paper's authors, but also by fellow (anonymous) researchers who reviewed previous drafts (provided in [12]). Yet, while it is objectively impossible to "universally confirm" this paper's overarching assumption, we provide supporting evidence in Section §7.3.

ASIA CCS '26, Bangalore, India

ACM ISBN 979-8-4007-2356-8/26/06
https://doi.org/10.1145/3779208.3807488

1) We will use prior work to provide a basis for the arguments and ASSERTIONS stated in this paper.
2) We will use prior work to suggest avenues for future work, or highlight exemplary good practices.
3) We will never explicitly criticize any prior work—barred those co-authored by this paper's co-authors.

In complying with the first two rules, we will draw from both recent literature, published in so-called "high-ranked venues" (such as [65], best paper award at EuroS&P'24); as well as from seminal works from the previous century—including some published when the concept of "NIDS" had not yet been conceived (such as the 1984 paper by Thompson on "Reflections on trusting trust" [125]).

Moreover, by adhering to the third rule, we will not attempt to "point the finger at" works which may not follow any of the assertions stated in this paper—except for those co-authored by the authors of this piece (such as [21]). Therefore, readers who expect to, e.g., find explicit mention of papers that "violate" any given assertion—even with the (legitimate) goal to motivate that there is a need for such an assertion—will be disappointed. We firmly believe that doing so is not constructive.

1.2 Target Audience and Contributions

This paper is addressed to any researcher with an interest in NIDS. We identify two classes of potential readers:

- People who want to carry out research in NIDS. This includes both the act of *designing* a plan to answer a given research question; or *writing* a document that summarizes or advances the state of the art; but also *reading* prior related work.
- People who serve as reviewers of NIDS-related documents. This includes both those who must gauge the scientific value of a given document (e.g., to determine whether the document should be "accepted" or "rejected" in a peer-reviewed selection procedure) and those who are solicited to provide feedback and constructive criticism on such a document to improve its quality.

Hence, this paper is meant to be understood by those with long-term expertise in NIDS, or by those who have tackled NIDS in orthogonal domains (e.g., applied ML, which is closely tied to NIDS as shown by Fig. 1), as well as by those who are just approaching any NIDS-related field. Therefore, we will favor simple logical arguments, rooted on elementary security principles, and we will deliberately not include complex notation or domain-specific jargon.

The contributions of this work are organized as follows. First, in Section §2, we define our scope and provide the essential terminology, through which we also clarify potential misconceptions. Then, in Sections §3 to §5, we state our **three assertions**—each focusing on a specific "issue" of NIDS-related research. We make some considerations that justify each assertion, and discuss implications and recommendations for future work. Next, in Section §6, we provide an exemplary demonstration of how our assertions can be used in NIDS-related research: we will do so with a **novel experiment**, through which we uncover results that substantially question prior knowledge in the field (our code is open source [12]). Lastly, in Section §7, we reflect on our assertions, clarifying potential misunderstandings, and also derive our **vademecum** (in Table 2). Conclusions are drawn in Section §8. The Appendix complements this piece with a glossary of NIDS (taken from the RFC [117]), and other real-world evidence supporting this paper's arguments.

2 Context and Preliminaries

To provide a constructive systematization that addresses the problem tackled by this paper, i.e., the "superficial treatment of the NIDS domain", we first define the scope of our work (§2.1). Then, we define some key terms that can be source of misunderstandings (§2.2). Finally, we conclude by describing a use-case of a NIDS (§2.3).

2.1 Scope: *Network* Intrusion Detection Systems

We focus on Network Intrusion Detection Systems, i.e., NIDS. This term is strongly related to the broader "Intrusion Detection System" (IDS), which has been extensively discussed in the last 38 years (e.g., [23, 35, 85, 129]), starting from the seminal work by Denning [57] in 1987.

The distinction between IDS and NIDS is the term "network", which can be understood in various ways. Among the most common interpretations, "network" can refer to the focus (or level) of the analysis—e.g., an IDS whose goal is to detect intrusion in network of hosts (in contrast to, e.g., an IDS which focuses on one host [31]); or to the data used to perform the detection—e.g., an IDS that analyzes network data [55] (in contrast to, e.g., host-level logs).

Here, we refer to an NIDS as an IDS that fulfills both of the aforementioned criteria. This is to define a clear separation with other types of IDS, such as those that focus on detecting intrusions at the host-level (e.g., HIDS) by using logs [66], source-code analysis [60], or provenance graphs (e.g., [16, 40, 48, 81, 93]); or even those that do use network-related data, but "centered" on a single host [13, 91].[2]

> We refer to a NIDS as *a system that focuses on detecting intrusions in a network by analysing network-related data.*

We report in the Appendix a glossary providing the actual definitions, taken verbatim from the RFC, of technical terms such as "system", "intrusion", "network". Importantly, we stress that the goal of an (N)IDS is to *detect intrusions*, i.e., security-noteworthy events that assume that an entity has obtained (or attempts to obtain) unauthorized access to a given system or resource. From this, we derive two important considerations: *(i)* the focus is to "detect" intrusions as early and effectively as possible to minimise damage—and not to prevent intrusions;[3] and *(ii)* to "detect an intrusion," it is implicit that the intrusion must have already occurred—in other words, (N)IDS operate with the assumption that attackers may have already acquired a foothold in the monitored network.

2.2 Terminology (and clarifications)

We discuss some terms associated with our interpretation of NIDS, in an attempt to prevent generating some misunderstandings (which we ourselves encountered), further clarify our scope, and provide some suggestions for future work.

Detection *approaches*. There exist various ways in which a NIDS can fulfill its detection objectives. Common terms are "signature-based" or "anomaly-based" approaches [23, 89, 120, 136]: the former detect intrusions by trying to identify known patterns of malicious

[2] Most assertions and recommendations made in this paper apply also to these complementary classes of IDS, as well as to any security system (e.g., those devoted to malware detection [15, 54, 126], or explainability [74], or protection of cyber-physical systems [47, 64, 69, 97], as well as those for the automotive context [44, 94, 95, 108]).
[3] We do acknowledge, however, that some (N)IDS may integrate some form of "prevention" mechanism that may automatically enact certain policies.

behaviour in the analysed data, whereas the latter seek to capture malicious events when there is some deviation from an established notion of normality. Both of these approaches have their pros and cons, and can be implemented either manually—by relying on heuristics, security feeds, organization-specific policies, or domain expertise; or via data-driven techniques—including those pertaining to the machine-learning (ML) domain, either supervised or unsupervised [45, 102, 119, 128, 133, 135, 137], as well as reinforcement-learning based (e.g., [21]); moreover, both signature- and anomaly-based approaches can work "online" (i.e., by analysing data in real time [70]) or "offline" (i.e., by analysing data in batches [77]). This paper covers all of these.

What is an *anomaly*? We emphasize that *an anomaly is not necessarily a security-noteworthy event*, and vice-versa. For instance, it is possible that a certain piece of software begins performing "novel" network activities which are benign in nature but are flagged as anomalies by the NIDS. Conversely, it is possible that a malicious host, in an attempt to evade an anomaly-based NIDS, performs network activities that mimic normal traffic patterns, therefore not triggering any anomaly by the NIDS. Such an observation is crucial: a NIDS that detects a lot of anomalies, even when these anomalies are "true" anomalies, is not necessarily a NIDS that works well from a security standpoint. In light of this, we endorse future work focusing on anomaly-based NIDS to clearly state what is the objective of the anomaly-detection mechanism.

***Evasion* of NIDS.** The term "evasion" can have many meanings in the security domain. Particularly, in the adversarial ML context, it is often used to denote attacks at "test time" [36], which can be misleading [19]. In this work, we define "evasion" in its literal sense. Given that we are considering a scenario in which there is an NIDS that inspects network traffic to detect intrusions in the network, an "evasion" can be considered as any circumstance in which an "intrusion" is not detected by the NIDS. In other words, our definition of evasion encompasses both that of "adversarial ML attacks" (in which a malicious sample is classified as benign via an adversarial perturbation introduced at test time [71]), as well as those achieved via, e.g., concealment attacks [64], and even those that are due to poor configuration of the NIDS [72]; the latter, however, cannot be considered as attacks because there is no deliberate intention by the attacker to evade the NIDS (see: RFC definitions of "attack" in Appendix A).

What is the *output* of an NIDS? A NIDS generates *alerts* (or *alarms*) [32, 56, 67, 115]: if the NIDS believes to have identified a security-noteworthy event, allegedly resembling an intrusion, then the NIDS raises one (or more) alerts. Such alerts are meant to be triaged by a human operator—potentially a member of the security operation center (SOC): if the alerts were truly representative of a security incident, then appropriate mitigations would be enacted; otherwise, the alarms would be considered as "false positives" and no action would be taken [14]. Nonetheless, even though NIDS may integrate some classification mechanisms that discriminate between benign and malicious datapoints, it is after post-processing the output of the NIDS (typically via automated means [128]) that an actual decision will be made. Abundant efforts now focus on developing "dashboards" that can facilitate the decision-making of SOC-analysts [68, 103] by visualizing the output of NIDS in actionable reports—a function fulfilled by so-called System Information and Event Management (SIEM) tools.

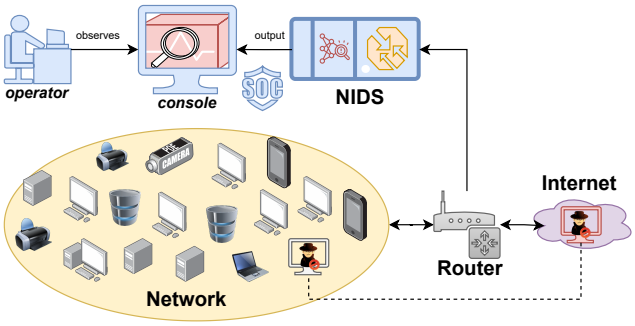

Fig. 2: Typical NIDS scenario. The NIDS receives data from the *router* as input, and shows its output in a *console*. The NIDS expects intrusions to occur in the network (or from the internet).

A comment on "unrealistic." In the security domain, it is common to encounter the term "unrealistic" (e.g., "this threat model is *unrealistic*"). We believe that this term is used with different meanings, which can lead to misunderstandings. In some cases, it is used to identify a scenario that is "unlikely to occur in reality"; in other cases, to identify a scenario that is "impossible to occur in reality". These two expressions are semantically very different: the latter is a clear indication of impossibility, whereas the former, despite being rather vague, does not exclude that a given scenario may occur. We advise readers to refrain from using this term on its own and without providing additional context.

2.3 Deployment Scenario of a NIDS

We provide in Fig. 2 a schematic of a typical NIDS deployment in an organization's network. Let us describe Fig. 2.

The network of an organization (yellow oval), contains a variety of heterogeneous devices. This network is connected to the Internet (purple cloud) via a border gateway router—which may potentially integrate some firewall-related network policies [52]. The router forwards all traffic that goes through it to the NIDS, which is positioned on security appliances that are part of the SOC (blue colored objects). The NIDS analyses the network-related data by following any given detection approach. Such network-related data can be in the form of, e.g., raw network-packet captures (i.e., PCAP), or statistical metadata summarizing the network communications occurring in the network (e.g., network flow records, i.e., NetFlow), or a combination thereof [23, 33, 50, 73, 83, 96, 102, 131].

According to the logic of the detection approach integrated in the NIDS, some alerts will be generated. Such alerts, typically after some post-processing (e.g., alert aggregation or clustering [128]), are presented on a dedicated *console* (potentially integrated in some SIEM) to the SOC operator—tasked to triage the threat reports that summarize the alerts, and enact the appropriate actions. Unfortunately, in the real world, the majority of these reports tend to be false positives [14]. Nevertheless, the figure also shows an attacker that has obtained unauthorized access to a host of the monitored network, which can be controlled remotely (e.g., via a remote-access trojan, or RAT). The NIDS may be able to detect such an intrusion if its detection approach can somehow identify its

malicious behaviour (e.g., [82]). If this does not happen, the attacker may expand his control on the targeted network (e.g., via lateral movement techniques [76]), or simply use their infected host as part of a botnet [78, 139], or passively exfiltrate data from such a host [112], or do nothing.

The aforementioned scenario (portrayed in Fig. 2) can be extended to cover also those cases in which, e.g., the NIDS is positioned in a subnetwork of the organization's network, and therefore the NIDS receives its input data from another routing appliance. It can also be positioned on a host that is running various virtual machines—and the NIDS is tasked to monitor the network operations of such virtual machines. Finally, the scenario can also envision a network of a small organization, which may not have a dedicated SOC and either outsources its security to a third-party company/vendor [25]; or only has a few IT/network experts that manage its security.

Given the above, and by applying elementary security principles, we can derive the following considerations:

- The NIDS, being a security-focused appliance (and being conceivably part of an SOC), is to be deployed in a highly secure location—potentially unreachable from the organization's network. We can expect the NIDS to be heavily protected by tight access-control mechanisms.
- The NIDS receives data from the router, and is tasked to analyse such data, looking for *intrusions occurring in the network*. In other words, the NIDS trusts the data received by the router (even if attackers attempt evasion). It is beyond the NIDS' scope to ensure the correct operation of the router.
- The console, being part of the NIDS, is protected by the same defensive layers of the NIDS, and its output is assumed to be observable only by highly-privileged users—such as SOC personnel or IT administrators.

These considerations are fundamental to understand our assertions.

3 Threat Modeling: attacks against NIDS

The first assertion focuses on a fundamental security aspect of attacks *against* NIDS: the threat model.

In designing (and describing) an attack against a NIDS (i.e., an "evasion" attempt), it is crucial to first define the characteristics of the envisioned attacker—i.e., a real person (or group) that is motivated to reach a specific goal by following a given strategy, which depends on the knowledge and capabilities of the attacker w.r.t. the targeted system [37].

> **Assertion 1**. It is non-sensical to assume a scenario in which a network intrusion detection system (NIDS) is expected to work against an attacker that has compromised such a system.

3.1 Considerations

The first assertion is rooted on a well-known fact: *a compromised security system is, by definition, not reliable*. We provide in Appendix B some excerpts, taken from notable prior works [17, 28, 100, 110, 134] (some over 40 years old [125]), supporting Assertion 1.

Before elucidating the implications of this assertion, let us align such an assertion to the NIDS context. Without loss of generality, let us assume an attacker that, after having obtained control of a host within an organization's network, wants to evade the NIDS protecting such a network.

According to Section 2, a NIDS receives network-related data as *input*—which is assumed to be trusted. Therefore, it is straightforward that the device (e.g., the router) that collects/forwards the data that the NIDS shall analyze must not have been compromised. If this is not the case (e.g., if the router sends "fake" network-related data to the NIDS), then it would be an overly-optimistic assumption to expect the NIDS to work against an attacker with such powers.

Moreover, after receiving some input data, the NIDS uses a variety of techniques to *analyse* such data. Therefore, it is straightforward that the data-analysis pipeline must not have been compromised. For instance, an attacker who can manipulate the internal operations of the NIDS could just default all communications involving the attacker-controlled hosts to not generate any alert.

Finally, the NIDS produces an *output* presented in a console. Therefore, the console must not have been compromised, too. For instance, if an attacker had write access to the console, then the attacker could delete the alerts raised by the NIDS; or, if the attacker had read access to the console, then they would be able to immediately discern if an alert is raised about their malicious activities, and change their tactic accordingly—before any action is taken by security operators.

Of course, attackers who have compromised the input data, analytical pipeline, or output console of the NIDS, may be capable of a variety of actions that can enable them to evade the NIDS. What we wrote above are just exemplary (and rather extreme) cases, whose purpose is to demonstrate, logically, that a NIDS becomes unreliable if any of these components has been compromised.

3.2 Implications and Recommendations

The principle of Assertion 1 is that, when designing an attack against a NIDS (i.e., an evasion attack), the researcher must ensure that the attack (or its implementation) does not implicitly assume a compromise of the NIDS itself.

For instance, by referencing the "adversarial ML" domain [30], wherein attackers try to reach their goals by crafting "adversarial perturbations", a perturbation applied in the "feature space" (see [107]) would represent a violation of Assertion 1, since it would imply that either the NIDS' input data (if, e.g., the NIDS analyses raw PCAP), or the data-processing pipeline (if, e.g., the NIDS analyses NetFlows), has been compromised; this was done in [21]. Similarly, precise manipulations of the "training dataset", used to train a given ML model used by the NIDS, could be seen as a compromise of the NIDS analytical pipeline, given that the logic of such an ML model depend on its training phase. Finally, threat models envisioning "black-box" attackers that can observe the output of the NIDS to refine their strategies would also imply that the NIDS has been compromised—and is, therefore, unreliable.

To comply with Assertion 1, we propose the following recommendation for future work seeking to study attacks against NIDS (or corresponding defenses):

> **Recommendation 1.** The threat model must assume attackers who can only control the hosts from the Internet, or the hosts within the network that they have compromised—which should not include the router that feeds data to the NIDS, the NIDS itself, or the NIDS output console.

The generic intuition is to avoid envisioning threat models in which the attacker has, in a sense, "already won". For instance, a valid evasion attempt can entail an attacker that, e.g., fragments the packets of their controlled hosts [17]. This would lead to completely different network-related data (e.g., PCAP) forwarded by the router—but would not imply a compromise of the router.[4]

Importantly, Assertion 1 is not to be intended as a constraint in the creation of novel threat models or, worse, a construct to criticize prior work. On the contrary, the goal is to induce a reflective exercise by the researcher: "if the attack's methodology *does* implicitly assume that the NIDS has been compromised, then *how can this be done*?" Put differently, the researcher should think of ways that would make the overarching attack not trivial. Otherwise, the researcher should acknowledge that the reason why the attack succeeds is (also) due to a compromise of the NIDS (or its subelements). In this case, however, the researcher should take into account the added cost to compromise the NIDS (e.g., how is it possible that the attacker manipulated the router to send fake data to the NIDS?), and potentially revise the attacker's goal (e.g., exfiltrate a large amount of sensitive data), or revise the attack's methodology (e.g., if the attacker can freely observe the output of the NIDS console, maybe there are better/cheaper alternatives to reach the same overarching objectives).

Note that assuming that the NIDS has been compromised is not ~~unrealistic~~ an impossible scenario. As written in a recent piece: "The reality is that systems will become compromised or will always have untrusted, partially trusted, or compromised elements" [109]. Therefore, there is reason even in studying circumstances that do not follow the aforementioned recommendation. For instance, to design a defense against a (very) powerful attacker.

4 Evaluation of NIDS: datasets and testbed

The second assertion focuses on a crucial aspect of most research papers: the experimental evaluation.

The de-facto way to verify whether a given claim holds (e.g., an attacker evades a NIDS) is to carry out an empirical assessment that, by using factual evidence, demonstrates that such a conclusion is "supported by the data."

Assertion 2. The evaluation testbed must account for the real-world characteristics and implications (for both "benign" and "malicious" hosts) of the NIDS' deployment scenario *envisioned in the research paper.*

4.1 Considerations

The second assertion is related to the "dataset" used to carry out the evaluation of NIDS-related research papers. Given our notion of NIDS (Section 2), such an evaluation necessitates a dataset containing network-related data.

The network-security domain has been historically known for being affected by an overall lack of publicly-available datasets that could be used to carry out meaningful experiments. This situation seemed to have improved in the last decade, with the release of various "benchmark" datasets that could be used by the research community [39]. However, as shown in the recent work by Flood et al. [65], widely-adopted datasets present significant inconsistencies that, in a sense, undermine the conclusions of papers reliant on them (including, e.g., [21]).

The problem, however, lies not in the benchmark datasets per-se—which are, ultimately, just data. Rather, the problem is in the fact that researchers may "blindly" rely on datasets containing network-related information that may not align with the scenario envisioned in the threat model. For instance, evaluating a NIDS meant to protect IoT devices on a dataset whose network traffic does not have any IoT device (see [122] for a summary) does not make a good argument—regardless of how "well-crafted" the dataset is (as also stated in [98]). At the same time, evaluating a NIDS on a dataset containing malicious traffic generated via methods that were popular decades ago, and showing that the NIDS can detect such attacks, is not a very convincing result. Besides, if the NIDS is expected to be deployed in a "network of a modern organization", it would be worrying if a similar organization is not protected against attacks for which known signatures exist.

Nonetheless, an important observation (orthogonal to the aforementioned problem) is that modern networks present immense variability [23, 119]. It is therefore overly-optimistic to expect that a research evaluation, even if carried out over (well-crafted) datasets that capture the behavior of diverse networks, can demonstrate that a given claim holds "in general" (i.e., anywhere/anytime).

4.2 Implications and Recommendations

It is straightforward that the best way to ensure an evaluation that complies with Assertion 2 is to carry out an experiment in a real-world network on which the researcher has (hypothetically) complete control and visibility: this way, the researcher can both *(i)* collect data encompassing a variety of "benign" activities—ideally pertaining to a large number of heterogeneous hosts; and *(ii)* reproduce a given set of "malicious" activities—ideally related to recent threats, and used to test the NIDS.

Unfortunately, carrying out such an evaluation may be beyond the reach of most academic researchers (due to, e.g., lack of permission to collect real-users' data, or deploying infected machines in an organization's network). In these cases, we propose the following recommendation:

Recommendation 2: Ensure that the deployment scenario described in the paper aligns with the network environment captured by the evaluation dataset.

The fundamental principle is to *avoid "overclaiming."*

There is nothing fundamentally wrong with using publicly available benchmark datasets. However, before carrying out an evaluation on a similar testbed, the researcher should first try to understand what the datasets contain (in terms of both benign and malicious network activities).

Regardless, relying *only* on benchmark datasets implicitly limits the scope of the evaluation—which can hardly resemble that of a "modern network" or "modern attacks". To overcome such a limitation, the researcher can: *(a)* create a dataset in a controlled environment—such as by using physical hardware (as done in, e.g., [102]) or virtual machines (as done in, e.g., [59]), or open-source toolkits for network-traffic generation, e.g., [49, 51, 113];

[4] Note that it is not trivial to replicate the effects of packet fragmentation by manipulating a pre-collected PCAP trace captured by the router [22].

and/or *(b)* review the most recent network threats, reproduce their behavior (e.g., by leveraging VulnHub [11]), and inject them in "benign" network activities [24, 26]—after ensuring that there are no network artifacts which may skew the final results [27]. Both of these approaches are: *(i)* scientifically valid, *(ii)* accessible to any researcher, *(iii)* enable some control on the testbed, and *(iv)* involve carrying out "real" network experiments. Alternatively, a researcher with access to real-world organizational data (which cannot be disclosed) can test a method on such data, and then validate it also on benchmarks: this is what has been done, e.g., in [105, 123].

Finally, and as a corollary to Assertion 2, we advocate that *reviewers* of NIDS papers refrain from overly emphasizing limitations that are inherently insurmountable. For instance, criticizing a paper because its evaluation cannot prove that a given result holds in general is not a constructive, and almost unfair, remark—unless the paper itself specifically claims generality. Otherwise, if the paper clearly defines its boundaries, then the crux is verifying if the evaluation's testbed aligns with the deployment scenario described in the paper—and then gauging whether an investigation carried out in such a scenario provides a significant scientific contribution.[5]

5 Performance Assessment: FPR and TPR

The third assertion focuses on a specific aspect of the empirical evaluation of an NIDS: the quantitative assessment of the NIDS performance and, in particular, of the false-positive rate (FPR) and true-positive rate (TPR).

Given that NIDS ultimately seek to "detect intrusions", such a task can be seen as a classification problem in which the goal is maximizing the TPR (i.e., correctly classifying "malicious" events) while minimizing the FPR (i.e., classifying "benign" events as malicious).

Assertion 3. Depending on the specific context, the TPR and the FPR can be incomplete performance metrics: they not necessarily are meaningful indicators of a NIDS' practical utility.

5.1 Considerations

The third assertion is strictly related to the base-rate fallacy problem [29]: modern networks generate such a large amount of data that some false alarms are bound to happen, leading to "alert fatigue" that overwhelms security operators, decreasing their efficiency in fulfilling their security duties [14, 101].

However, from the viewpoint of a SOC operator, it is not necessary to inspect all alerts, e.g., raised by a given NIDS' submodule. Consider, for instance, the results achieved by Engelen et al. [62] on the "fixed" version of a well-known dataset containing a mix of benign and malicious NetFlows (collected via simulations lasting around one working week). Specifically, the dataset included 1 823 964 benign and 881 211 malicious NetFlows: 75% of these have been used to train a classifier, which has been tested on the remaining 25%; hence, the test set included 455 991 benign samples, and 220 302 malicious samples. Looking at the classification results reported in [62], the classifier correctly detected ≈219 492 malicious samples, and ≈451 431 benign samples,[6] meaning an FPR of ≈0.01 with a TPR of >0.99. Now, setting aside the TPR and FPR, we argue that *it is inconceivable to think that a human operator can manually triage the over 220k samples deemed as malicious by the classifier.*

This is not a matter of the FPR or TPR being low or high. Even if the FPR was 0, and the TPR was 1.0, no human would triage 220k alerts—or, at least, not one-by-one. Recall (from Section 2.2) that, in the real-world, NIDS are typically part of larger systems (e.g., SIEM) that aggregate the output of various modules into a comprehensive report that enables security analysts to quickly take action. (E.g., see Figs. 4 or 5 in the Appendix A.) For instance, the recent work by Van Ede et al. [128] clearly shows that practitioners apply post-processing techniques to cluster the plethora of alerts raised by intrusion-detection modules, resulting in more manageable threat reports that can be manually triaged (to some degree).

Note that the aforementioned problem (which affects, e.g., [21]) is orthogonal to that of reporting just the FPR without providing the overall number of samples—which can also be source of confusion [17, 25]. For instance, very little can be inferred by reporting that a given classifier achieves an FPR of 0.01 without mentioning how many datapoints are analysed by such a classifier.

5.2 Implications and Recommendations

The crux of Assertion 3 is that NIDS produce (a lot of) alerts which *may* be indicators of just a handful of "intrusions" affecting one (or more) hosts.

Hence, we propose the following recommendation—addressed at future work seeking to propose/scrutinize NIDS-related modules (e.g., ML-based classifiers):

Recommendation 3: the performance of a NIDS-related module should be tied to its practical utility—which requires taking into account also post-processing techniques applied to the output of such a module, and the overarching operational context.

The underlying idea of this recommendation is *don't stop.*

Consider a paper whose claimed contribution is a NetFlow classifier. Such a classifier can be integrated in a NIDS by pre-processing the raw PCAP into NetFlows, and then having the classifier analyse such NetFlows (which are deemed to be either benign or malicious). Instead of stopping here and simply measuring the TPR and FPR, the researcher should go one step further, and apply post-processing techniques (potentially proposed by prior work, e.g., [128]) to the NetFlows deemed malicious by the classifier. It is at this point that the TPR and FPR should be computed: out of all the "clusters of threats" detected by the NIDS *thanks to the classifier*, how many were truly security-noteworthy events?

For instance, let us re-examine the use-case discussed in Section 5.1. From the experiments by Engelen et al. [62], it could have been inferred that the majority of malicious samples entailed the same pair of (compromised) hosts. In this case, even if a classifier, different from the one used in [62], missed some malicious samples (thereby achieving a lower TPR w.r.t. [62]), the overall number of raised alarms could still enable the identification of an intrusion

[5] **Example.** Let us consider the CICIDS17 dataset [116], captured in a network of a dozen hosts over the course of one working week. Using such a dataset is valid if it is stated that, e.g., "we consider a small-scale network of a dozen hosts which generate ≈50GB of network traffic over five days."

[6] These numbers have been obtained by looking at TABLES I and II of [62]. For TABLE II, the metrics are provided with only two decimal points, so there may be slight discrepancies in the overall counting. We stress that [62]'s contribution was fixing a previously proposed dataset.

(meaning that its overall utility would be the same as that of [62]). It could also be that, in a non-critical context, a classifier having, on the surface, a significantly lower TPR (due to missing many malicious NetFlows) could have a higher utility if, e.g., it can detect at least some malicious NetFlows concealing a very subtle intrusion. At the same time, a classifier raising thousands of "false positives" may not be a problem in practice if such alarms are all related to the same host, resulting in a single "false report" that would not cause excessive burden to a security analyst.

In short, we advocate that the FPR and TPR are assessed by referring to what the analyst would observe by looking at the security console—and not by focusing solely on a single component of the overarching NIDS.[7]

6 Demonstration (Novel Experiments)

In what follows, we showcase a practical way to use our Assertions in a research context. To this purpose, we carry out novel experiments that leverage well-known and publicly available resources, guaranteeing complete reproducibility of our results (our experimental source code is provided in our repository [12]).

6.1 Setting and Threat Model

This subsection focuses on applying Assertion 1.

Considered network. We envision a scenario of a small-scale (and non-critical [87]) network comprised of a dozen of hosts. Such hosts encompass a variety of operating systems (OS), spanning across both Windows and UNIX-based OSs. These hosts can communicate with hosts from "external" networks, which can be reached through a router which also integrates some firewall mechanism. The router, alongside forwarding the traffic to the respective hosts, is also tasked to automatically generate the network flows (NetFlows) of the corresponding traffic. The NetFlows produced by the router are sent to a classifier that is tasked to infer if they are "benign" or "malicious" (i.e., a simple binary classification task). In particular, the classifier has been trained so that it can discriminate legitimate NetFlows from malicious NetFlows belonging to a variety of well-known attacks, such as: SSH/FTP-bruteforcing (via `patator` [7]); DoS attempts (via `slowloris`[8], `slowhttptest` [4], `Hulk` [2], `GoldenEye` [3]) ; as well as XSS, SQL Injections, the `HeartBleed` [10] OpenSSL exploit, and some specific Infiltration attempts related to DropBox. Prior work has shown that NetFlows are ideal to detect similar threats [138].

Security system. The network is protected by a NIDS. The NIDS receives as input the NetFlows produced by the router, as well as the output of the classifier. These results are shown to a network operator who must make a decision on whether to investigate an internal host that is (potentially) subject of an attack, or not. Due to the non-critical and small-scale nature of the network, human-checks are carried out in batches and on a daily basis.

Attacker. We assume an attacker who gained access to some of the hosts of the network (e.g., via some zero-day exploit, or some unpatched vulnerability [75]). The attacker wants to cause some damage to this network. The attacker is aware that the network is protected by some NIDS that integrates a classifier trained on samples of the aforementioned attacks. The attacker is not aware of the exact training data, and is also not aware of the specifics of the classifier. The attacker cannot make any query to, e.g., see the output of the classifier. To achieve her objective, the attacker relies on attacks that are not within the training set of the classifier: the DDos `LOIT` [9], and malware related to the `ARES` botnet [5].

Compliance with Assertion 1. The aforementioned scenario describes an hypothetical setting that matches the assumptions of NIDS. The attacker has gained a foothold in the network, and the NIDS must detect the attacker's presence. The attacker has compromised only hosts within the network, but has no control on any component within the overarching security system.

6.2 Implementation and Pre-Assessment

Here, we show how to comply with Assertion 2.

Dataset. One way to assess the envisioned scenario is by using the well-known CICIDS17 dataset [116]. Indeed, looking at the description of this dataset, the scenario matches perfectly: the data was captured in a network with < 20 hosts over the course of five days. On the last day, the creators of CICIDS17 launched attacks conforming to `LOIT` and `ARES`, whereas all other attacks are captured on the previous days. Therefore, a good way to practically reproduce our envisioned scenario by using CICIDS17 is to train an ML model on the data of the first four days, and then use the data of the fifth day as test (such data also includes benign traffic, which is fundamental for false positives). Such an experimental protocol follows best practices (i.e., no temporal snooping [18, 23, 27]).

Development. We are aware that the CICIDS17 is flawed [65], so we will use the variant provided by Engelen et al. [62]. To further align our setup with prior work, we will use the open-source code in [23], wherein a complete re-assessment of existing ML-based methods has been carried out—and which also encompasses the CICIDS17 dataset. Practically, we proceed as follows:

(1) we take the PCAP data of CICIDS17 [116], extract the NetFlows and label them according to [62], and apply the pre-processing steps of [23], which distinguish internal-from-external hosts and also standardize the ports according to the IANA groups [1], as well as subsampling data (extracting ≈500k benign NetFlows, and at most 133k malicious NetFlows per attack type).
(2) We split the data into train and test: all NetFlows generated on the last day of the capture are considered as test, whereas all the rest is considered as training. Such a split yields ≈342k NetFlows in the test set (of which ≈87k are benign, denoting the large number of malicious NetFlows of the DDoS attack), and 588k in the train set (of which ≈409k are benign).
(3) We train a variety of ML-based classifiers: decision tree (DT), random forest (RF), histogram-gradient boosting (HGB), logistic regression (LR).[9] All these classifiers support binary classification. Before training each classifier, we split the training set into train:validation with an 80:20 split (note that [23] found no statistically-significant difference in using different types

[7] Practically, this can be done by implementing some post-processing modules that, e.g., "if a host generates more than [threshold] malicious samples, then raise an alarm" [99]. Clustering mechanisms can also be employed to aggregate similar datapoints [128]. In both of these cases, two classifiers having different TPR may have the same utility as they would both raise the same number of alerts.

[8] https://github.com/gkbrk/slowloris

[9] Of course, in practice the NIDS would use only one classifier, but for the sake of this experiment we consider many to show the importance of Assertion 3

Table 1: Case Study Results. We show the results of our novel experiment. We report the TPR, FPR, and total number of errors (misclassifications) achieved by our four considered classifiers when they processed the validation set and the test set at the "NetFlow-level"; and then show the TPR and TPR when the focus is on identifying internal hosts involved in malicious communications (which we do by post-processing the output of the classifiers).

	Validation set (NetFlows)			Test set (NetFlows)			Test set (Hosts)		
Classifier	TPR	FPR	#Errors	TPR	FPR	#Errors	TPR	FPR	#Errors
DT	0.9996	0.0003	45	0.0636	0.0002	238,807	1.000	0.250	2
RF	0.9996	<0.0001	16	0.3706	0.0000	160,515	0.125	0	7
HGB	0.9998	<0.0001	9	0.3729	<0.0001	159,913	0.125	0.125	8
LR	0.9373	0.0711	8,068	0.3734	0.0564	164,722	1.000	1.000	8

of split on this dataset). The intention is to validate the performance of these classifiers before their deployment: if their performance (on data "similar" to that used to train them) is subpar, then such classifiers would not be deployed in practice. Afterwards, we will test the classifiers on the test set.

Preliminary assessment. Our classifiers perform well on the validation set. The DT achieved a TPR of 0.9996 and a FPR of 0.0003 (45 total misclassifications). The RF achieved a TPR of 0.9996 and a FPR of <0.0001 (16 total misclassifications). The HGB achieved a TPR of 0.9998 and a FPR of <0.0001 (9 total misclassifications). The LR achieved a TPR of 0.9373 and a FPR of 0.0711 (8,068 total misclassifications). Note that these results align with those in [23] (even though this specific experiment had not been carried out in [23]). Moreover, we can see that the LR appears to be the worst classifier of the group. The best choice – by looking at the FPR and TPR – is the HGB (which is also the fastest to train, only 6s).

Compliance with Assertion 2. Our experimental setup aligns with our envisioned scenario. We will not attempt to generalize our results: whatever finding we will derive from our experiments, we will not claim that it can be extended to enterprise networks or to IoT scenarios (neither of these are covered by CICIDS17).

6.3 Results and Post-processing

Here, we show how to comply with Assertion 3.

TPR and FPR on the test set. We assess our classifiers the test set. We can expect that the performance, especially the TPR, will be low: this is because the attacks in the test set stem from generative processes different from those on which the classifiers have been trained. However, one of the promises of ML-based classifiers is to "detect unseen attacks" [25, 119], so we hope that at least some samples will be detected—especially given that the classifiers are trained on some form of DDoS attack. The results are as follows. For the DT, TPR=0.0636 and FPR=0.0002 (238,807 total misclassifications). For the RF, TPR=0.3706 and FPR=0 (160,515 total misclassifications). For the HGB, TPR=0.3729 and FPR=<0.0001 (159,913 total misclassifications). For the LR, TPR=0.3734 and FPR=0.0564 (164,722 total misclassifications). From these results, we can derive that the "best" classifier at test time is either RF or HGB. Indeed, LR has the highest TPR, but the FPR is very high; the DT has the worst TPR of the group; HGB and RF are similar: RF has 0 FPR which is appreciable, but the HGB has also a very low FPR and a slightly higher TPR.

A reflection. Drawing conclusions based on the results above would be misleading. Recall that a NIDS' output is meant to be observed by an operator (§2) who would make a choice. If the network operator monitoring this network would see the output of the aforementioned classifiers, what would they do? Even the classifier that raised the lowest number of "positives", the DT, predicted that 16,247 NetFlows are malicious. A human analyst cannot manually triage all such NetFlows to make a decision. Indeed, we quote a statement we made in describing our envisioned scenario (emphasis ours): "These results are shown to a network operator who must make a decision on whether to investigate an *internal host that is (potentially) subject of an attack*, or not." Hence, what our envisioned operator would do is check which internal hosts are included in NetFlows that have been flagged as malicious by the classifier.

The real performance. To examine the workflow of our envisioned operator, we developed a simple module that reports how many internal hosts have been involved in the malicious NetFlows flagged by each classifier. For reference, a classifier having accuracy of 100% would flag that 8 internal hosts should be investigated (whereas the remaining 8 hosts that made some communications on this day should not). The results are as follows.

- The DT flagged 10 hosts: 8 are involved in malicious NetFlows (i.e., the "true positives"), whereas 2 are not (i.e., "false positives").
- The RF flagged a single host, which was involved in malicious communications. However, it missed all the other 7 hosts.
- The HGB flagged two hosts: one was indeed involved in malicious communications, the other was not.
- The LR flagged 16 hosts: 8 were indeed involved in malicious communications, but 8 were not.

So, in practice, if the operator were observing the output of the DT, then the TPR would be 1.000 (because 8 out of 8 hosts-that-should-be-investigated would be truly investigated) and the FPR 0.250 (because the operator would inspect two hosts that did not have any malicious activity). Conversely, if the operator relied on the RF, then the TPR would be 0.125 and the FPR=0; and for the HGB, the TPR would also be 0.125, but the FPR=0.125. Finally, for the LR, both TPR and FPR would be 1. From this perspective, the DT appears to be much better than both the HGB and the RF—even though the performance of the DT was statistically-significantly (p<.05) lower than both of them on the test set (from the viewpoint of individual NetFlows classification) and also lower in the validation set.

Compliance with Assertion 3. Instead of stopping at the results of individual classifiers, we went one step further and examined how the classifier's output could be integrated in an operational context. By adopting a different perspective and post-processing the output, we found that a classifier that appeared to be suboptimal (classification-wise) may be the best (utility-wise).

We conclude with a remark. We do not claim that our results hold in general. And we also do not claim that DT-based classifiers are always better than RF-/HGB-based ones. We also do not claim that our post-processing module is the one adopted in the real-world,

nor that security operators would behave exactly in the same way we envisioned in our case study. We reiterate that this is a SoK paper meant for researchers. We provide the complete results in Table 1 (the code in our open-source repository [12]).

7 Discussion and Related Work

We critically examine the main contributions of this work, i.e., the assertions and their recommendations (§7.1); we also expand our discussion with practical observations (§7.2). Then, we provide some anecdotes serving as a motivation for our work (§7.3). In doing so, we also cover some related works. Finally, we provide our vademecum and takeaways (§7.4).

7.1 Critical Remarks (Frequently Asked Questions)

To avoid generating misunderstandings, we discuss potential counterarguments to the Assertions made in this paper. We do so by adopting a question and answer format.

"*With regards to 'adversarial perturbations', wouldn't Assertion 1 imply that studying any form of adversarial ML attack is pointless in the NIDS context?*" No. As stated in §3.1, Assertion 1 is not meant to be "a constraint in the creation of novel threat models or, worse, a construct to criticize prior work." Although *some* adversarial ML attacks envisioned by prior work (e.g., [21]) may have entailed "unrealizable" perturbations—potentially stemming from attackers that had (implicitly) already "breached" the NIDS—this does not undermine the value of any such prior (or even future) publication.

- First, because it is not wrong in a *research paper* to make assumptions that do not fully align with the real world.
- Second, because any "misalignment" may not be apparent at the time of publication (after all, research papers are at the forefront of human knowledge), and can be used as a lesson learned for future research (as an example: [27]).
- Third, because even an ~~unrealistic~~ unlikely assumption (e.g., an attacker who has compromised the overarching security system) may still occur in reality.
- Fourth, because even if the likelihood of such an occurrence is low, from a research viewpoint it is still insightful to study some security/robustness properties of any given NIDS component to investigate worst-case scenarios.

Nevertheless, we stress that it is possible to stage adversarial ML attacks against NIDS that do not assume a compromise of the overarching security system. For instance, attacks at *training-time* (so-called "poisoning") can be launched just from a single attacker-controlled host—we strongly recommend looking into the threat model envisioned in the recent work by Severi et al. [114]; whereas attacks involving adversarial perturbations crafted at *inference-time* require a careful treatment of the "problem space"[10] but are certainly plausible. For noteworthy examples, we point the reader to the recent papers by Erba et al. [63] and Catillo et al. [42], both of which discussing ways attackers can use to influence the data seen by ML models—while complying with Assertion 1.

"*Many works have examined prior literature (on various forms of intrusion detection) under a real-world lens. What is the added value of Assertion 2, then?*" It is true that many papers (e.g., [20, 23, 27, 43, 45, 65, 106]) have, explicitly or implicitly, scrutinized the testbed or evaluation methodology of related works from a realistic viewpoint. However, the takeaways of all of these contributions are orthogonal to that of Assertion 2. Specifically, we focus the attention to the recommendation written in §4.2: we argue that, in principle, it is scientifically acceptable for a research paper to carry out an evaluation in a "toy" testbed—as long as the paper does not claim that the results hold in the real world. However, we also argue that given the impossibility of replicating the behavior of "all" real-world networks (or to test the effectiveness of an NIDS "in general"), reviewers should lower their expectations—and not use the fact that a paper carries out an evaluation via a (potentially well-done) simulation as a "weakness" of a paper just because "simulations cannot replicate the real world". We are not aware of any work (in the intrusion detection context) that advocated for such recommendations—which, we stress, are addressed to reviewers, too (and hence pertain to "metascience" research).[11]

"*For Assertion 3, wouldn't neglecting the FPR/TPR prevent one from measuring the value of a specific NIDS submodule (which can be a paper's main contribution)?*" This is a valid remark—which is, however, orthogonal to the point of Assertion 3. Specifically, the point is not to "neglect" the TPR or FPR (which *can* be appropriate performance metrics); rather, the point is that, in some cases, it may be more appropriate to focus on the real-world utility of a given NIDS submodule (e.g., a classifier). Such utility can only be measured by taking into account the overall system performance. At the same time, and from the viewpoint of a peer-reviewer, it may be unfair to criticize the FPR (or TPR) for being "too high" (or "too low"): such critiques should be done by accounting for the role played by these metrics in the overarching NIDS—and not by merely looking at the results of a single classifier (in our experiment, the TPR of the DT was the worst on the NetFlows, but near-perfect when identifying hosts involved in malicious communications—see Table 1.) Rather than critizing the low/high TPR/FPR, a reviewer may suggest to discuss how the FPR/TPR of a classifier can impact the system-wide effectiveness of the envisioned NIDS.

"*Must any future work follow/comply with the three assertions stated in this paper to provide a meaningful contribution to the state of the art?*" No. The scientific contribution of any research paper (past or future) on NIDS is independent from how well it embraces any of the assertions proposed in our work. Ultimately, ours is a reflective piece: researchers are free to disagree with the statements made in this work. However, we also believe that the assertions (and corresponding recommendations) proposed in this piece provide some original viewpoints—addressed also to researchers serving as peer-reviewers—that can be beneficial for this research domain (such as, e.g., not recommending to "reject" a paper because there is no evaluation on a real enterprise network).[12]

7.2 Practical Observations

Here, we provide some complementary observations, rooted on established prior work and operational best practices, that expand

[10] For instance, it is crucial to identify how a real-world attacker can, from their controlled hosts, perform actions that affect the feature representation of a sample that is ultimately analysed by (and then evades) an ML model.

[11] An argument can be made about the NIDS vs malware-detection domains in peer-review contexts. For the latter, we believe that research is more aligned with the real world (e.g., [46, 84]), because real-world malware and goodware examples are easier to find and use for research. In contrast, NIDS-related data is closed-source, and experiments require simulations which are easy to criticize (our view is in §4.2).

[12] Reviewers should be more mindful of how NIDS relate to the real world.

the contributions of this piece. The intention is to inspire future developments in the NIDS domain from a research viewpoint.[13]

Attacker's Knowledge. Assertion 1 predominantly focuses on the *capabilities* of the attacker. However, the *knowledge* of the attacker, which is a crucial component of an adversarial threat model, is orthogonal to the capabilities [19]. For instance, an attacker can have "perfect knowledge" of a system without having compromised such a system: this can happen if, e.g., an open-source implementation of such a system is available and a company uses such a prototype in production (see, e.g., [104]). However, and as remarked in [20], having complete knowledge of the entire system is challenging from the viewpoint of a real-world attacker, since it would require insider's knowledge—which is feasible, but costly. According to [20], the most likely use-case of attackers trying to bypass an NIDS is that of a "partial-knowledge adversary", who may reasonably infer (potentially via OSINT) some information about the NIDS (or the underlying network environment) used by a targeted company. In summary, we believe that making sensible assumptions on the attacker's knowledge to be important, but complementary to Assertion 1.

Approximating Real-world Scenarios. As we stated for Assertion 2 (§4.1), some researchers may not have access to data from real-world networks for experiments. This, however, should not prevent one from carrying out evaluations that *resemble* real-world scenarios. We believe that, for research papers, even a proof-of-concept (but well-executed) simulation to be a valid way to assess certain claims (e.g., [53]). Especially recently, many works proposed tools for network-traffic simulations (e.g., [113, 127, 130, 132]) usable to mimic real-world setups—potentially by enriching existing benchmark datasets with more recent datapoints [130]. Nonetheless, sharing the entire artifacts (including, e.g., documentation on how the simulation was carried out—and not just the data itself) can substantially improve the quality of an experimental evaluation, since downstream research can determine how well the testbed used in prior work can be used as a basis for new experiments.

Detection Timeliness. In some contexts (e.g., critical infrastructures), alerts should be triaged immediately, and malicious communications should be found as early as possible. In these cases, a classifier with perfect TPR (assuming low FPR) is desirable because it would detect "all" malicious attempts—which may lead to an earlier response by human operators (who should triage such alarms in near-real time). However, the sheer number of events generated in modern infrastructures is massive [124], making it unfeasible to triage every single alert (e.g., the authors of DeepCase [128] consider "throttling" periods varying between 15-minutes and 1 day). Nevertheless, we therefore solicit researchers to *clearly state how important the timeliness of the detection is for the envisioned scenario* (to support Assertion 3). For instance, in our demonstration (§6), we explicitly stated that the human analysts triages alarms on a daily basis—which is a sensible assumption given that we envisioned a small-scale and non-critical network.

Usages of a Classifier's Output. A corollary of Assertion 3 is that the output of a classifier (e.g., whether an input is benign or malicious) can be itself processed by additional models and methods (see Section 5.2). Our demonstrative experiment (in Section 6.3) showed a simple use case of "aggregator of host-specific detections", which was meant to be provided to a human analyst for triaging. However, the pipeline between classifier output and human decision can be made up of various components that can cross-correlate information derived from various sources and thus reduce the number of alerts that must be manually triaged [34, 79]. For instance, a recent work [80] combines alerts with threat-intelligence; whereas [41] combine alerts from multiple sensors; while [111] combine outputs of multiple defensive schemes. Put simply, there are numerous ways in which the output of a given component (such as a NetFlow classifier) can be integrated in an operational security system. Hence, the quality of a component should not be solely assessed in isolation, but by accounting for the overarching system's performance and the respective deployment context. From the viewpoint of a researcher (and to echo our Recommendation-3 in Section 5.2), we thus endorse exploring all these possibilities—which would be facilitated by disclosure of open-source prototypes.

7.3 Motivation

As stated in the Introduction (§1.1), the contributions of this work are built on the assumption that "research in NIDS-related areas is relatively stagnant, and is hindered by—among others—a superficial treatment of the NIDS domain."

In what follows, we provide factual evidence suggesting that such an assumption holds (as of September 2025). In doing so, we maintain the same rationale followed in the rest of the paper: we *are not criticizing any prior work*.

- During the interactive author-reviewer discussion[14] of a NeurIPS'23 paper on NIDS [90], a reviewer pointed out that not enough information was provided on the features used to train the ML models. The authors responded that "*Considering that NeurIPS focuses on AI, we didn't include too much details on network traffic in our paper.*" Such details were later added in the final version of the paper. ⇒ While this is an example of a constructive peer-review, this anecdote suggests that in some (top-tier and ML-centered) research venues, the NIDS domain may be treated superficially.
- Many prior works (e.g., [14, 14, 23, 101]) have explicitly pointed out the skepticism, or uncertainty, of practitioners towards techniques proposed in NIDS-related research. And this is surprising given the thousands of publications on this subject (see Fig. 1). For instance, false alarms were well-known even >25 years ago (see, e.g., [29, 56]) and are still widespread today (see, e.g., [14, 101]). However, the concept of "alarm" (or of "alert") may not be among the priorities of some NIDS-related researches (e.g., in [21], the terms "alarm" or "alert" or even "report" are *never mentioned*[15]), despite being intrinsic of NIDS (see §2.2). ⇒ These anecdotes further show that *(i)* some fundamental characteristics of NIDS may be overlooked by NIDS-related papers, and *(ii)* from a practical viewpoint, research advances in this domain are lackluster.
- One of the most popular benchmark datasets for NIDS is CICIDS17, as well as its extension the CICIDS18: according to their creators (see [6, 8]), both of these datasets refer to the same research paper [116] which counted 5 962 citations on Google

[13] These observations have been derived during the peer-review phase of this paper.

[14] Publicly observable at: https://openreview.net/forum?id=zfCNwRQ569

[15] For "report", we are referring to its usage as a noun (there are plenty of occurrences of "report" as a verb in [21]). In contrast, in [21] there is mention of "false positive" which, as argued in Assertion 3, it can be a misleading (or incomplete) metric.

Scholar since its publication in 2018 and until the end of 2025. However, in 2021, a work by Engelen et al. [62] (published in the Workshop on Traffic Measurements for Cybersecurity, co-located with the IEEE Symposium on Security and Privacy) carried out a thorough analysis of CICIDS17 and *(i)* found substantial issues while also *(ii)* releasing the code to "fix" such issues. In 2022, Liu et al. [92] found and fixed errors in CICIDS18 (in a work that won the Best Paper Award at IEEE CNS). To provide additional evidence that *some* research papers may overlook the real-world implications of NIDS-related data, we study the "citation trend" of these three works, visually shown in Fig. 3. Indeed, it is reasonable to assume that *(a)* a paper using CICIDS17/18 would cite [116]; and *(b)* researchers *aware* of the issues of CICIDS17/18 that would still use this dataset for their experiments would cite [62] or [92].[16] Therefore, in a perfect world, the citation trend of [62, 92] and [116] would align.[17] However, we can see from Fig. 3 that this is not the case: since 2022 and until the end of 2025, the works by Engelen et al. [62] and Liu et al. [92] cumulatively accrued 457 citations, whereas the publication of CICIDS17/18 [116] obtained 4 578 citations during the same timespan. ⇒ This result shows that prior work may overlook the issues affecting the real-world validity of the data used to test a given NIDS-related hypothesis.[18]

- Expanding on the point above, even in top-tier conferences both *(i)* authors and *(ii)* program-committee members may not be aware of the issues of CICIDS17 (which have been known since 2021). For instance, a recent IEEE S&P'24 paper (i.e., [58]) carried out an evaluation on CICIDS17 but without accounting for the findings of Engelen et al. [62] (which was presented at a workshop of IEEE S&P!). Of course, and as a disclaimer, this does not mean that any conclusion based on the "original" CICIDS17 is unfounded. For instance, let us consider the author-reviewer discussion[19] of a NeurIPS'24 paper on NIDS [38]. It was pointed out that the initial submission carried out its evaluation on the original CICIDS17: the authors promptly (and remarkably) re-did their experiments on the "fixed" version of CICIDS17, and found that the results still supported the conclusions. ⇒ Put simply, the point is that there is nothing fundamentally wrong in using the "original" CICIDS17 for NIDS-related research; however, focusing excessively on the "research" side of a publication (without, e.g., questioning what is contained in a given benchmark dataset) may hinder some real-world implications of its conclusions.

To conclude, we believe that all of the above is evidence of some fundamental misunderstandings of the NIDS domain—which may (perhaps unconsciously) affect both authors of research papers, as well as peer-reviewers. As we wrote (see §1.2), our paper is addressed at both of these categories of readers. Authors and reviewers can look at some of the arguments made here and use them to reflect on their statements (either in their paper, or in the review).

[16] Indeed, if a paper used CICIDS17 (or CICIDS18) but does not cite [62] (or [92]), there is a high risk that the evaluation is carried out on the "flawed" data of CICIDS17 (or CICIDS18) without even acknowledging potential threat to validity.

[17] There are other works that discussed issues in CICIDS17 or CICIDS18, such as [65, 88], but they received even less citations than [62, 92], hence we omit them from this analysis as our conclusions would not be affected.

[18] Note that it would be unfair to question the ecological validity of papers (such as [118]) considering CICIDS17 but which have been published before [62].

[19] Publicly observable at: https://openreview.net/forum?id=KYHVBsEHuC

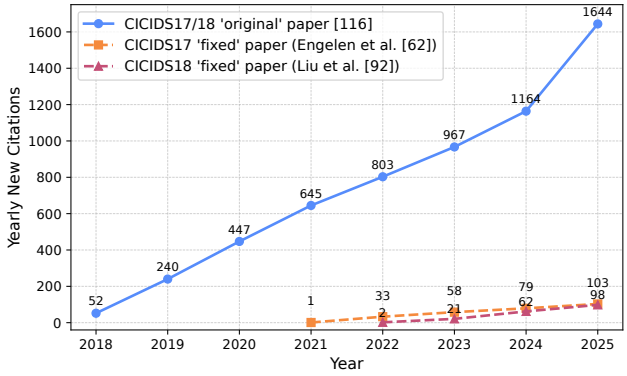

Fig. 3: Comparison between [116] and [62, 92]. We plot the new citations achieved by [116] (original paper of CICIDS17/18) and the work by Engelen et al. [62] (paper that "fixed" CICIDS17) and Liu et al. [92] (paper that "fixed" CICIDS18) every year since their publication (Source: Google Scholar).

7.4 Takeaways and Vademecum

We have touched a variety of topics pertaining to NIDS—which we endorse researchers to take into account whenever they engage with this domain. To facilitate this, we coalesce our assertions and recommendations into a set of seven questions, shown in Table 2. These questions represent the *vademecum* mentioned in the Introduction (§1.1). Let us briefly discuss each question (Q).

- Q1 and Q2 do not refer to a specific Assertion, but are fundamental to avoid triggering misunderstandings.
- Q3 and Q4, related to Assertion 1, are particularly useful for adversarial ML papers.
- Q5 and Q6, related to Assertion 2, serve to keep the paper's findings safely anchored on its testbed (which should be carefully examined, especially if publicly-available data is used).
- Q7, related to Assertion 3, serves as a reminder to try adopting the viewpoint of a security operator, i.e., the real user of a NIDS.

Ideally, these questions can be useful to both *(i)* investigators/authors of papers, or *(ii)* reviewers. The former can use these questions either at the beginning of a given research project (e.g., to shape a given research plan), or during the course of the research activities, as well as during the writing of an article (e.g., as "sanity checks"). The latter can use these questions to guide the reviewing process of a submission. Potentially, these questions can expedite the author-reviewer interactions, or help clarifying some doubts.[20]

In our demonstration (§6), we followed our vademecum: "anomaly" and "unrealistic" have never been used; the attacker is not assumed to have compromised the NIDS; the dataset was carefully chosen so as to resemble the envisioned scenario; there is no "generality claim"; and the TPR/FPR have been assessed by considering the post-processed output of a classifier.

8 Conclusions and Future Outlook

We seek to induce a change in the landscape of NIDS research.

The assertions stated in this work are rooted in established security principles and may appear well-known to some readers; similarly, some recommendations also echo those made in security-focused literature. For instance, the seminal work by Sommer and

[20] We would be delighted if the authors of another submission under review can "convince" a referee (e.g., during a rebuttal) by referencing this work!

Table 2: Vademecum. Questions to ask when approaching NIDS research (Ref. points to this SoK's section). Some questions may not be universally applicable.

#	Question	Ref.
Q1	Has the notion of "anomaly" been properly defined?	§2.2
Q2	Has the term "unrealistic" been used unambiguously?	§2.2
Q3	Does the evasion attack's methodology *implicitly* assume a compromise of the NIDS (or of its input/output)?	§3
Q4	(if yes to Q3) Has the fact that the NIDS is assumed to be compromised been taken into account, or at least justified?	§3.2
Q5	Does the evaluation dataset capture the characteristics of the network environment envisioned in the threat model?	§4
Q6	Does the paper claim "generality" of the conclusions—and, if so, is there sufficient evidence to substantiate such a claim?	§4.2
Q7	Has the TPR/FPR been operationally contextualized (e.g., how does the analyst use the output of a classifier)?	§5.2

Paxson [119], published over 15 years ago, emphasized, e.g., the necessity of realistic threat models in the NIDS research domain. However, we believe that the takeaways of [119] may have faded. Our assertions and vademecum, supported by our original experiment, hence reinforces the messages broadcast by [119].

We hope that the research community on NIDS can be inspired by the arguments and reflective exercises provided in this SoK.

Acknowledgments

The author would like to thank all the reviewers who provided feedback to previous versions of this manuscript. Parts of this research was funded by Hilti.

References

[1] 2011. Internet Assigned Numbers Authority. https://www.iana.org/assignments/service-names-port-numbers/service-names-port-numbers.txt.
[2] 2012. Hulk. https://github.com/grafov/hulk.
[3] 2014. GoldenEye. https://github.com/jseidl/GoldenEye.
[4] 2016. slowhttptest. https://github.com/shekyan/slowhttptest.
[5] 2017. Ares. https://github.com/sweetsoftware/Ares.
[6] 2017. Intrusion detection evaluation dataset (CIC-IDS2017). https://www.unb.ca//cic/datasets/ids-2017.html.
[7] 2017. Patator. https://github.com/lanjelot/patator.
[8] 2018. CSE-CIC-IDS2018 on AWS. https://www.unb.ca//cic/datasets/ids-2018.html.
[9] 2018. Loic. https://www.cloudflare.com/learning/ddos/ddos-attack-tools/low-orbit-ion-cannon-loic/.
[10] 2020. HeartBleed. https://cheese-hub.github.io/secure-coding/03-heartbleed/index.html.
[11] 2024. Vulnhub. https://github.com/vulhub/vulhub.
[12] 2026. Repository of this paper. https://github.com/hihey54/asiaccs26_sok.
[13] Cristina Abad, Jed Taylor, Cigdem Sengul, William Yurcik, Yuanyuan Zhou, and Ken Rowe. 2003. Log correlation for intrusion detection: A proof of concept. In *ACSAC*.
[14] Bushra A Alahmadi, Louise Axon, and Ivan Martinovic. 2022. 99% False Positives: A Qualitative Study of {SOC} Analysts' Perspectives on Security Alarms. In *USENIX SEC*.
[15] Hisham Alasmary, Aminollah Khormali, Afsah Anwar, Jeman Park, Jinchun Choi, Ahmed Abusnaina, Amro Awad, Daehun Nyang, and Aziz Mohaisen. 2019. Analyzing and detecting emerging Internet of Things malware: A graph-based approach. *IEEE Internet of Things Journal* 6, 5 (2019), 8977–8988.
[16] Abdulellah Alsaheel, Yuhong Nan, Shiqing Ma, Le Yu, Gregory Walkup, Z Berkay Celik, Xiangyu Zhang, and Dongyan Xu. 2021. {ATLAS}: A sequence-based learning approach for attack investigation. In *30th USENIX security symposium (USENIX security 21)*. 3005–3022.
[17] Ross J Anderson. 2001. *Security engineering: a guide to building dependable distributed systems*.
[18] Giuseppina Andresini, Feargus Pendlebury, Fabio Pierazzi, Corrado Loglisci, Annalisa Appice, and Lorenzo Cavallaro. 2021. Insomnia: Towards concept-drift robustness in network intrusion detection. In *ACM workshop on Artificial Intelligence and Security*. 111–122.
[19] Giovanni Apruzzese, Hyrum S Anderson, Savino Dambra, David Freeman, Fabio Pierazzi, and Kevin Roundy. 2023. "Real Attackers Don't Compute Gradients": Bridging the Gap Between Adversarial ML Research and Practice. In *SaTML*.
[20] Giovanni Apruzzese, Mauro Andreolini, Luca Ferretti, Mirco Marchetti, and Michele Colajanni. 2021. Modeling realistic adversarial attacks against network intrusion detection systems. *ACM Digital Threats: Research and Practice* (2021).
[21] Giovanni Apruzzese, Mauro Andreolini, Mirco Marchetti, Andrea Venturi, and Michele Colajanni. 2020. Deep reinforcement adversarial learning against botnet evasion attacks. *IEEE Transactions on Network and Service Management* (2020).
[22] Giovanni Apruzzese, Aurore Fass, and Fabio Pierazzi. 2024. When adversarial perturbations meet concept drift: an exploratory analysis on ml-nids. In *ACM AISec*.
[23] Giovanni Apruzzese, Pavel Laskov, and Johannes Schneider. 2023. Sok: Pragmatic assessment of machine learning for network intrusion detection. In *IEEE EuroS&P*.
[24] Giovanni Apruzzese, Luca Pajola, and Mauro Conti. 2022. The cross-evaluation of machine learning-based network intrusion detection systems. *IEEE TNSM* (2022).
[25] Giovanni Apruzzese et al. 2022. The Role of Machine Learning in Cybersecurity. *ACM DTRAP* (2022).
[26] Ignacio Arnaldo and Kalyan Veeramachaneni. 2019. The Holy Grail of "Systems for Machine Learning" Teaming humans and machine learning for detecting cyber threats. *ACM SIGKDD Explorations Newsletter* (2019).
[27] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. 2022. Dos and don'ts of machine learning in computer security. In *USENIX Security*.
[28] Remzi H Arpaci-Dusseau and Andrea C Arpaci-Dusseau. 2018. *Operating systems: Three easy pieces*.
[29] Stefan Axelsson. 2000. The base-rate fallacy and the difficulty of intrusion detection. *ACM Transactions on Information and System Security (TISSEC)* (2000).
[30] Md Ahsan Ayub, William A Johnson, Douglas A Talbert, and Ambareen Siraj. 2020. Model evasion attack on intrusion detection systems using adversarial machine learning. In *2020 54th annual conference on information sciences and systems (CISS)*. 1–6.
[31] Rebecca Bace and Peter Mell. 2001. Intrusion Detection Systems. *NIST Special Publication on Intrusion Detection Systems* (2001).
[32] Tao Ban, Takeshi Takahashi, Samuel Ndichu, and Daisuke Inoue. 2023. Breaking alert fatigue: AI-assisted SIEM framework for effective incident response. *Applied Sciences* 13, 11 (2023), 6610.
[33] Diogo Barradas, Nuno Santos, Luís Rodrigues, Salvatore Signorello, Fernando MV Ramos, and André Madeira. 2021. FlowLens: Enabling Efficient Flow Classification for ML-based Network Security Applications.. In *NDSS*.
[34] Mohan Baruwal Chhetri, Shahroz Tariq, Ronal Singh, Fatemeh Jalalvand, Cecile Paris, and Surya Nepal. 2024. Towards human-AI teaming to mitigate alert fatigue in security operations centres. *ACM Transactions on Internet Technology* 24, 3 (2024), 1–22.
[35] Elmarie Biermann, Elsabe Cloete, and Lucas M Venter. 2001. A comparison of intrusion detection systems. *Computers & Security* (2001).
[36] Battista Biggio, Igino Corona, Davide Maiorca, Blaine Nelson, Nedim Šrndić, Pavel Laskov, Giorgio Giacinto, and Fabio Roli. 2013. Evasion attacks against machine learning at test time. In *ECMLKDD*.
[37] Battista Biggio and Fabio Roli. 2018. Wild patterns: Ten years after the rise of adversarial machine learning. *Pattern Recognition* (2018).
[38] Abdullah Bin Jasni, Akiko Manada, and Kohei Watabe. 2024. DiffuPac: Contextual Mimicry in Adversarial Packets Generation via Diffusion Model. *NeurIPS* (2024).
[39] Philipp Bönninghausen, Rafael Uetz, and Martin Henze. 2024. Introducing a Comprehensive, Continuous, and Collaborative Survey of Intrusion Detection Datasets. In *Cyber Secur. Exp. and Test Workshop*.
[40] Robert A Bridges, Tarrah R Glass-Vanderlan, Michael D Iannacone, Maria S Vincent, and Qian Chen. 2019. A survey of intrusion detection systems leveraging host data. *ACM CSUR* (2019).
[41] Blake D Bryant and Hossein Saiedian. 2020. Improving SIEM alert metadata aggregation with a novel kill-chain based classification model. *Computers & Security* 94 (2020), 101817.
[42] Marta Catillo, Antonio Pecchia, Antonio Repola, and Umberto Villano. 2024. Towards realistic problem-space adversarial attacks against machine learning in network intrusion detection. In *ARES*.
[43] Marta Catillo, Antonio Pecchia, and Umberto Villano. 2023. Machine learning on public intrusion datasets: Academic hype or concrete advances in NIDS?. In

DSN-S.
[44] Paolo Cerracchio, Stefano Longari, Michele Carminati, Stefano Zanero, et al. 2024. Investigating the impact of evasion attacks against automotive intrusion detection systems. In *Symposium on Vehicles Security and Privacy (VehicleSec)*.
[45] Fabrício Ceschin, Marcus Botacin, Albert Bifet, Bernhard Pfahringer, Luiz S Oliveira, Heitor Murilo Gomes, and André Grégio. 2024. Machine learning (in) security: A stream of problems. *DTRAP* (2024).
[46] Fabrício Ceschin, Marcus Botacin, Heitor Murilo Gomes, Luiz S Oliveira, and André Grégio. 2019. Shallow security: On the creation of adversarial variants to evade machine learning-based malware detectors. In *ROOTS*.
[47] Pin-Yu Chen, Shin-Ming Cheng, and Kwang-Cheng Chen. 2012. Smart attacks in smart grid communication networks. *IEEE Communications Magazine* 50, 8 (2012), 24–29.
[48] Zijun Cheng, Qiujian Lv, Jinyuan Liang, Yan Wang, Degang Sun, Thomas Pasquier, and Xueyuan Han. 2024. Kairos: Practical intrusion detection and investigation using whole-system provenance. In *IEEE Symposium on Security and Privacy (SP)*.
[49] Henry Clausen, Robert Flood, and David Aspinall. 2019. Traffic generation using containerization for machine learning. In *Workshop on DYnamic and Novel Advances in Machine Learning and Intelligent Cyber Security*.
[50] Carlos Garcia Cordero, Sascha Hauke, Max Mühlhäuser, and Mathias Fischer. 2016. Analyzing flow-based anomaly intrusion detection using replicator neural networks. In *IEEE PST*.
[51] Carlos Garcia Cordero, Emmanouil Vasilomanolakis, Aidmar Wainakh, Max Mühlhäuser, and Simin Nadjm-Tehrani. 2021. On generating network traffic datasets with synthetic attacks for intrusion detection. *ACM Transactions on Privacy and Security* (2021).
[52] Jordan Cropper, Johanna Ullrich, Peter Frühwirt, and Edgar Weippl. 2015. The role and security of firewalls in iaas cloud computing. In *ARES*.
[53] Levente Csikor, Himanshu Singh, Min Suk Kang, and Dinil Mon Divakaran. 2021. Privacy of DNS-over-HTTPS: Requiem for a Dream?. In *2021 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE Computer Society, 252–271.
[54] Savino Dambra, Yufei Han, Simone Aonzo, Platon Kotzias, Antonino Vitale, Juan Caballero, Davide Balzarotti, and Leyla Bilge. 2023. Decoding the secrets of machine learning in malware classification: A deep dive into datasets, feature extraction, and model performance. In *CCS*.
[55] Hervé Debar, Marc Dacier, and Andreas Wespi. 1999. Towards a taxonomy of intrusion-detection systems. *Computer networks* (1999).
[56] Hervé Debar and Andreas Wespi. 2001. Aggregation and correlation of intrusion-detection alerts. In *RAID*.
[57] Dorothy E Denning. 1987. An intrusion-detection model. *IEEE TSE* (1987).
[58] Alec F Diallo and Paul Patras. 2024. Sabre: Cutting through Adversarial Noise with Adaptive Spectral Filtering and Input Reconstruction. In *IEEE S&P*.
[59] Christian Dietz, Raphael Labaca Castro, Jessica Steinberger, Cezary Wilczak, Marcel Antzek, Anna Sperotto, and Aiko Pras. 2018. IoT-botnet detection and isolation by access routers. In *2018 9th International Conference on the Network of the Future (NOF)*. IEEE, 88–95.
[60] Manuel Egele, Martin Szydlowski, Engin Kirda, and Christopher Kruegel. 2006. Using static program analysis to aid intrusion detection. In *DIMVA*.
[61] Elastic. 2025. SIEM from Elastic. https://web.archive.org/web/20250221173307/https://www.elastic.co/security/siem.
[62] Gints Engelen, Vera Rimmer, and Wouter Joosen. 2021. Troubleshooting an intrusion detection dataset: the CICIDS2017 case study. In *IEEE S&PW*.
[63] Alessandro Erba, Andres F Murillo, Riccardo Taormina, Stefano Galelli, and Nils Ole Tippenhauer. 2024. On Practical Realization of Evasion Attacks for Industrial Control Systems. In *Proceedings of the 2024 Workshop on Re-design Industrial Control Systems with Security*.
[64] Alessandro Erba, Riccardo Taormina, Stefano Galelli, Marcello Pogliani, Michele Carminati, Stefano Zanero, and Nils Ole Tippenhauer. 2020. Constrained concealment attacks against reconstruction-based anomaly detectors in industrial control systems. In *ACSAC*.
[65] Robert Flood, Gints Engelen, David Aspinall, and Lieven Desmet. 2024. Bad design smells in benchmark nids datasets. In *EuroS&P*.
[66] Anderson Frasão, Tiago Heinrich, Vinicius Fulber-Garcia, Newton C Will, Rafael R Obelheiro, and Carlos A Maziero. 2024. I See Syscalls by the Seashore: An Anomaly-based IDS for Containers Leveraging Sysdig Data. In *ISCC*.
[67] Clement Fung, Eric Zeng, and Lujo Bauer. 2024. Attributions for ML-based ICS anomaly detection: From theory to practice. In *Proc. 31st Netw. Distrib. Syst. Secur. Symp.*
[68] Gustavo González-Granadillo, Susana González-Zarzosa, and Rodrigo Diaz. 2021. Security information and event management (SIEM): Analysis, trends, and usage in critical infrastructures. *Sensors* (2021).
[69] David Grochocki, Jun Ho Huh, Robin Berthier, Rakesh Bobba, William H Sanders, Alvaro A Cárdenas, and Jorjeta G Jetcheva. 2012. AMI threats, intrusion detection requirements and deployment recommendations. In *2012 IEEE Third International Conference on Smart Grid Communications (SmartGridComm)*. IEEE, 395–400.
[70] Eric Gyamfi and Anca Delia Jurcut. 2022. Novel online network intrusion detection system for industrial IoT based on OI-SVDD and AS-ELM. *IEEE Internet of Things Journal* (2022).
[71] Dongqi Han, Zhiliang Wang, Ying Zhong, Wenqi Chen, Jiahai Yang, Shuqiang Lu, Xingang Shi, and Xia Yin. 2021. Evaluating and improving adversarial robustness of machine learning-based network intrusion detectors. *IEEE Journal on Selected Areas in Communications* (2021).
[72] Mark Handley, Vern Paxson, and Christian Kreibich. 2001. Network Intrusion Detection: Evasion, Traffic Normalization, and {End-to-End} Protocol Semantics. In *USENIX SEC*.
[73] Ahmad Hariri, Murat Yuksel, and David Mohaisen. 2024. RL-Based Speculative Installation of Unseen Flows in SDNs for Low-Latency Applications. In *2024 IEEE International Conference on Machine Learning for Communication and Networking (ICMLCN)*. IEEE, 250–256.
[74] Yiling He, Jian Lou, Zhan Qin, and Kui Ren. 2023. Finer: Enhancing state-of-the-art classifiers with feature attribution to facilitate security analysis. In *CCS*.
[75] Hwanjo Heo and Seungwon Shin. 2018. Who is knocking on the telnet port: A large-scale empirical study of network scanning. In *Proceedings of the 2018 on Asia Conference on Computer and Communications Security*. 625–636.
[76] Grant Ho, Mayank Dhiman, Devdatta Akhawe, Vern Paxson, Stefan Savage, Geoffrey M Voelker, and David Wagner. 2021. Hopper: Modeling and detecting lateral movement. In *USENIX Security Symposium*.
[77] Pedro Horchulhack, Eduardo K Viegas, and Altair O Santin. 2022. Toward feasible machine learning model updates in network-based intrusion detection. *Computer Networks* (2022).
[78] Idriss Idrissi, Mohammed Boukabous, Mostafa Azizi, Omar Moussaoui, and Hakim El Fadili. 2021. Toward a deep learning-based intrusion detection system for IoT against botnet attacks. *IAES International Journal of Artificial Intelligence* (2021).
[79] Fatemeh Jalalvand, Mohan Baruwal Chhetri, Surya Nepal, and Cecile Paris. 2024. Alert prioritisation in security operations centres: A systematic survey on criteria and methods. *Comput. Surveys* 57, 2 (2024), 1–36.
[80] Michał Jaworski and Bahareh Pahlevanzadeh. 2026. Hybrid Intelligence Endpoint Defense (HIED): A Data-Fusion Approach for Proactive Malware Detection. In *2026 IEEE 5th International Conference on AI in Cybersecurity (ICAIC)*. IEEE, 1–8.
[81] Zian Jia, Yun Xiong, Yuhong Nan, Yao Zhang, Jinjing Zhao, and Mi Wen. 2024. {MAGIC}: Detecting advanced persistent threats via masked graph representation learning. In *33rd USENIX Security Symposium (USENIX Security 24)*. 5197–5214.
[82] Dan Jiang and Kazumasa Omote. 2015. An approach to detect remote access trojan in the early stage of communication. In *IEEE international conference on advanced information networking and applications*.
[83] Shota Kawanaka, Yoshikatsu Kashiwabara, Kohei Miyamoto, Masazumi Iida, Chansu Han, Tao Ban, Takeshi Takahashi, and Jun'ichi Takeuchi. 2023. Packet-Level Intrusion Detection Using LSTM Focusing on Personal Information and Payloads. In *2023 18th Asia Joint Conference on Information Security (AsiaJCIS)*. IEEE, 88–94.
[84] Yigitcan Kaya, Yizheng Chen, Marcus Botacin, Shoumik Saha, Fabio Pierazzi, Lorenzo Cavallaro, David Wagner, and Tudor Dumitraş. 2025. ML-Based Behavioral Malware Detection Is Far From a Solved Problem. In *SaTML*.
[85] Ansam Khraisat, Iqbal Gondal, Peter Vamplew, and Joarder Kamruzzaman. 2019. Survey of intrusion detection systems: techniques, datasets and challenges. *Cybersecurity* (2019).
[86] Edward Klein. 2025. The Top 5 Open-Source NIDS Solutions. https://web.archive.org/web/2/https://logz.io/blog/5-open-source-nids/.
[87] Eric D Knapp. 2024. *Industrial Network Security: Securing critical infrastructure networks for smart grid, SCADA, and other Industrial Control Systems*. Elsevier.
[88] Maxime Lanvin, Pierre-François Gimenez, Yufei Han, Frédéric Majorczyk, Ludovic Mé, and Éric Totel. 2022. Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes. In *International Conference on Risks and Security of Internet and Systems*. Springer, 18–33.
[89] Seunghyeon Lee, Jinwoo Kim, Seungwon Shin, Phillip Porras, and Vinod Yegneswaran. 2017. Athena: A framework for scalable anomaly detection in software-defined networks. In *IEEE DSN*.
[90] Ruoyu Li, Qing Li, Yu Zhang, Dan Zhao, Yong Jiang, and Yong Yang. 2023. Interpreting unsupervised anomaly detection in security via rule extraction. *Advances in Neural Information Processing Systems* (2023).
[91] Jinxin Liu, Murat Simsek, Burak Kantarci, Mehran Bagheri, and Petar Djukic. 2022. Collaborative feature maps of networks and hosts for ai-driven intrusion detection. In *IEEE Global Communications Conference*.
[92] Lisa Liu, Gints Engelen, Timothy Lynar, Daryl Essam, and Wouter Joosen. 2022. Error prevalence in nids datasets: A case study on cic-ids-2017 and cse-cic-ids-2018. In *2022 IEEE conference on communications and network security (CNS)*. IEEE.

[93] Ming Liu, Zhi Xue, Xianghua Xu, Changmin Zhong, and Jinjun Chen. 2018. Host-based intrusion detection system with system calls: Review and future trends. *ACM CSUR* (2018).

[94] Yangyang Liu, Lei Xue, Sishan Wang, Xiapu Luo, Kaifa Zhao, Pengfei Jing, Xiaobo Ma, Yajuan Tang, and Haiying Zhou. 2025. Vehicular Intrusion Detection System for Controller Area Network: A Comprehensive Survey and Evaluation. *IEEE Transactions on Intelligent Transportation Systems* (2025).

[95] Stefano Longari, Daniel Humberto Nova Valcarcel, Mattia Zago, Michele Carminati, and Stefano Zanero. 2020. CANnolo: An anomaly detection system based on LSTM autoencoders for controller area network. *IEEE Transactions on Network and Service Management* (2020).

[96] Chun-Nan Lu, Yuan-Cheng Lai, Chun-Ying Huang, and Ying-Dar Lin. 2016. Hardware design for statistical network traffic classifiers. In *2016 2nd IEEE International Conference on Computer and Communications (ICCC)*. IEEE, 2425–2429.

[97] Yuan Luo, Ya Xiao, Long Cheng, Guojun Peng, and Danfeng Yao. 2021. Deep learning-based anomaly detection in cyber-physical systems: Progress and opportunities. *ACM Computing Surveys (CSUR)* 54, 5 (2021), 1–36.

[98] Zuchao Ma, Liang Liu, Weizhi Meng, Xiapu Luo, Lisong Wang, and Wenjuan Li. 2023. ADCL: toward an adaptive network intrusion detection system using collaborative learning in IoT networks. *IEEE Internet of Things Journal* 10, 14 (2023), 12521–12536.

[99] Marcello Meschini, Giorgio Di Tizio, Marco Balduzzi, and Fabio Massacci. 2024. A Case-Control Study to Measure Behavioral Risks of Malware Encounters in Organizations. *IEEE TIFS* (2024).

[100] Microsoft. 2025. *Trusted Compute Base.* Technical Report. https://learn.microsoft.com/en-us/azure/confidential-computing/trusted-compute-base

[101] Jaron Mink, Hadjer Benkraouda, Limin Yang, Arridhana Ciptadi, Ali Ahmadzadeh, Daniel Votipka, and Gang Wang. 2023. Everybody's got ML, tell me what else you have: Practitioners' perception of ML-based security tools and explanations. In *IEEE S&P*.

[102] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. 2018. Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection. In *NDSS*.

[103] Adabi Raihan Muhammad, Parman Sukarno, and Aulia Arif Wardana. 2023. Integrated security information and event management (siem) with intrusion detection system (ids) for live analysis based on machine learning. *Procedia Computer Science* (2023).

[104] Milad Nasr, Yanick Fratantonio, Luca Invernizzi, Ange Albertini, Loua Farah, Alex Petit-Bianco, Andreas Terzis, Kurt Thomas, Elie Bursztein, and Nicholas Carlini. 2025. Evaluating the Robustness of a Production Malware Detection System to Transferable Adversarial Attacks. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. 4394–4408.

[105] Samuel Ndichu, Tao Ban, Takeshi Takahashi, and Daisuke Inoue. 2023. Machine learning–based security alert screening with focal loss. In *2023 IEEE International Conference on Big Data (BigData)*. IEEE, 3043–3052.

[106] Feargus Pendlebury, Fabio Pierazzi, Roberto Jordaney, Johannes Kinder, and Lorenzo Cavallaro. 2019. {TESSERACT}: Eliminating experimental bias in malware classification across space and time. In *USENIX security symposium*.

[107] Fabio Pierazzi, Feargus Pendlebury, Jacopo Cortellazzi, and Lorenzo Cavallaro. 2020. Intriguing properties of adversarial ml attacks in the problem space. In *IEEE S&P*.

[108] Francesco Pollicino, Dario Stabili, and Mirco Marchetti. 2024. Performance comparison of timing-based anomaly detectors for controller area network: A reproducible study. *ACM Transactions on Cyber-Physical Systems* (2024).

[109] Awais Rashid, Sana Belguith, Matthew Bradbury, Sadie Creese, Ivan Flechais, and Neeraj Suri. 2025. Beyond Security-by-design: Securing a compromised system. *arXiv:2501.07207* (2025).

[110] Peter Reiher. 2018. *Operating systems: Three easy pieces.* Arpaci-Dusseau Books, LLC Madison, WI, USA, Chapter Introduction to Operating System Security.

[111] Tom-Martijn Roelofs, Eduardo Barbaro, Svetlana Pekarskikh, Katarzyna Orzechowska, Marta Kwapień, Jakub Tyrlik, Dinu Smadu, Michel Van Eeten, and Yury Zhauniarovich. 2024. Finding harmony in the noise: Blending security alerts for attack detection. In *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*. 1385–1394.

[112] Bushra Sabir, Faheem Ullah, M Ali Babar, and Raj Gaire. 2021. Machine learning for detecting data exfiltration: A review. *ACM CSUR* (2021).

[113] Xabier Sáez-de Cámara, Jose Luis Flores, Cristóbal Arellano, Aitor Urbieta, and Urko Zurutuza. 2023. Gotham testbed: a reproducible IoT testbed for security experiments and dataset generation. *IEEE Transactions on Dependable and Secure Computing* (2023).

[114] Giorgio Severi, Simona Boboila, Alina Oprea, John Holodnak, Kendra Kratkiewicz, and Jason Matterer. 2023. Poisoning network flow classifiers. In *ACSAC*.

[115] Ankit Shah, Rajesh Ganesan, and Sushil Jajodia. 2019. A methodology for ensuring fair allocation of CSOC effort for alert investigation. *IJIS* (2019).

[116] Iman Sharafaldin, Arash Habibi Lashkari, Ali A Ghorbani, et al. 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSp* (2018).

[117] Robert Shirey. 2007. *Internet security glossary, version 2.* Technical Report.

[118] Mahdi Soltani, Behzad Ousat, Mahdi Jafari Siavoshani, and Amir Hossein Jahangir. 2023. An adaptable deep learning-based intrusion detection system to zero-day attacks. *JISA* (2023). (the preprint appeared in 2021: https://arxiv.org/abs/2108.09199).

[119] Robin Sommer and Vern Paxson. 2010. Outside the closed world: On using machine learning for network intrusion detection. In *IEEE Symp. Secur. Privacy*.

[120] Wenjia Song, Hailun Ding, Na Meng, Peng Gao, and Danfeng Yao. 2024. Madeline: Continuous and low-cost monitoring with graph-free representations to combat cyber threats. In *2024 Annual Computer Security Applications Conference (ACSAC)*. IEEE, 874–889.

[121] Splunk. 2025. Security Software & Solutions. https://www.splunk.com/en_us/products/cyber-security.html.

[122] Carlotta Tagliaro, Martina Komsic, Andrea Continella, Kevin Borgolte, and Martina Lindorfer. 2024. Large-Scale Security Analysis of Real-World Backend Deployments Speaking IoT-Focused Protocols. In *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses*. 561–578.

[123] Anum Talpur, Jannik Schröder, Liliana Kistenmacher, Georg Becker, Wolfram Wingerath, and Mathias Fischer. 2025. Tagging Alerts to Adversaries: ML-Enabled Classification Using MITRE ATT&CK. In *IEEE CNS*.

[124] Shahroz Tariq, Mohan Baruwal Chhetri, Surya Nepal, and Cecile Paris. 2025. Alert fatigue in security operations centres: Research challenges and opportunities. *Comput. Surveys* 57, 9 (2025), 1–38.

[125] Ken Thompson. 1984. Reflections on trusting trust. *Commun. ACM* (1984).

[126] Jianwen Tian, Wei Kong, Debin Gao, Tong Wang, Taotao Gu, Kefan Qiu, Zhi Wang, and Xiaohui Kuang. 2025. Density Boosts Everything: A One-stop Strategy for Improving Performance, Robustness, and Sustainability of Malware Detectors.. In *NDSS*.

[127] Rafael Uetz, Christian Hemminghaus, Louis Hackländer, Philipp Schlipper, and Martin Henze. 2021. Reproducible and adaptable log data generation for sound cybersecurity experiments. In *Proceedings of the 37th annual computer security applications conference*. 690–705.

[128] Thijs Van Ede, Hojjat Aghakhani, Noah Spahn, Riccardo Bortolameotti, Marco Cova, Andrea Continella, Maarten van Steen, Andreas Peter, Christopher Kruegel, and Giovanni Vigna. 2022. Deepcase: Semi-supervised contextual analysis of security events. In *IEEE Symp. Secur. Privacy*.

[129] Emmanouil Vasilomanolakis, Shankar Karuppayah, Max Mühlhäuser, and Mathias Fischer. 2015. Taxonomy and survey of collaborative intrusion detection. *ACM CSUR* (2015).

[130] Miel Verkerken, Laurens D'hooge, Bruno Volckaert, Filip De Turck, and Giovanni Apruzzese. 2026. ConCap: Practical Network Traffic Generation for (ML-and) Flow-based Intrusion Detection Systems. In *Proceedings of the 4th IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*.

[131] Gernot Vormayr, Joachim Fabini, and Tanja Zseby. 2020. Why are my flows different? A tutorial on flow exporters. *IEEE Comm. Surv. Tut.* (2020).

[132] Ping-Lun Wang, Tzu-Wei Chao, Chia-Chien Wu, and Hsu-Chun Hsiao. 2022. Tool: an efficient and flexible simulator for byzantine fault-tolerant protocols. In *2022 52nd annual IEEE/IFIP international conference on dependable systems and networks (DSN)*. IEEE, 287–294.

[133] Yung-Chung Wang, Yi-Chun Houng, Han-Xuan Chen, and Shu-Ming Tseng. 2023. Network anomaly intrusion detection based on deep learning approach. *Sensors* (2023).

[134] James J. Whitmore. 2001. A method for designing secure solutions. *IBM Systems Journal* (2001).

[135] Limin Yang, Wenbo Guo, Qingying Hao, Arridhana Ciptadi, Ali Ahmadzadeh, Xinyu Xing, and Gang Wang. 2021. {CADE}: Detecting and explaining concept drift samples for security applications. In *USENIX Security Symposium*.

[136] Zhen Yang, Xiaodong Liu, Tong Li, Di Wu, Jinjiang Wang, Yunwei Zhao, and Han Han. 2022. A systematic literature review of methods and datasets for anomaly-based network intrusion detection. *Computers & Security* (2022).

[137] Stefano Zanero and Sergio M Savaresi. 2004. Unsupervised learning techniques for an intrusion detection system. In *ACM SAC*.

[138] Changwang Zhang, Zhiping Cai, Weifeng Chen, Xiapu Luo, and Jianping Yin. 2012. Flow level detection and filtering of low-rate DDoS. *Computer Networks* 56, 15 (2012), 3417–3431.

[139] Junjie Zhang, Roberto Perdisci, Wenke Lee, Unum Sarfraz, and Xiapu Luo. 2011. Detecting stealthy P2P botnets using statistical traffic fingerprints. In *2011 IEEE/IFIP 41st International Conference on Dependable Systems & Networks (DSN)*. IEEE, 121–132.

Appendix A Glossary and Images

We provide a list of technical definitions, taken verbatim from the RFC [117], which are crucial for this paper.

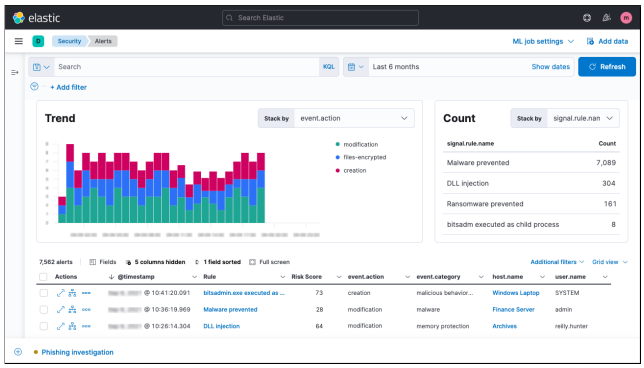

Fig. 4: The dashboard of Elastic – source: [61]

- *Attack*: An intentional act by which an entity attempts to evade security services and violate the security policy of a system. That is, an actual assault on system security that derives from an intelligent threat.
- *(Computer) Network*: A collection of host computers together with the subnetwork or internetwork through which they can exchange data. This definition is intended to cover systems of all sizes and types, ranging from the complex Internet to a simple system composed of a personal computer dialing in as a remote terminal of another computer.
- *(Information) System*: An organized assembly of computing and communication resources and procedures – i.e., equipment and services, together with their supporting infrastructure, facilities, and personnel – that create, collect, record, process, store, transport, retrieve, display, disseminate, control, or dispose of information to accomplish a specified set of functions.
- *Intrusion*: A security event, or a combination of multiple security events, that constitutes a security incident in which an intruder gains, or attempts to gain, access to a system or system resource without having authorization to do so. (alternate) A type of threat action whereby an unauthorized entity gains access to sensitive data by circumventing a system's security protections.
- *Intrusion Detection System*: A process or subsystem, implemented in software or hardware, that automates the tasks of *(a)* monitoring events that occur in a computer network and *(b)* analyzing them for signs of security problems. (alternate) A security alarm system to detect unauthorized entry.
- *Security Compromise*: A security violation in which a system resource is exposed, or is potentially exposed, to unauthorized access.

We provide in Figs. 4, 5, and 6 some exemplary dashboards of state-of-the-art NIDS software (i.e., SIEM).

Appendix B Excerpts supporting Assertion 1

Many seminal works support Assertion 1, and many software houses leverage its underlying a principle to design their solutions. Below are some excerpts (taken verbatim, emphasis ours) that justify Assertion 1.

- "The Trusted Computing Base (TCB) refers to all of a system's hardware, firmware, and software components that provide a secure environment. The components inside the TCB are considered 'critical.' *If one component inside the TCB is compromised, the entire system's security may be jeopardized.*" [100]

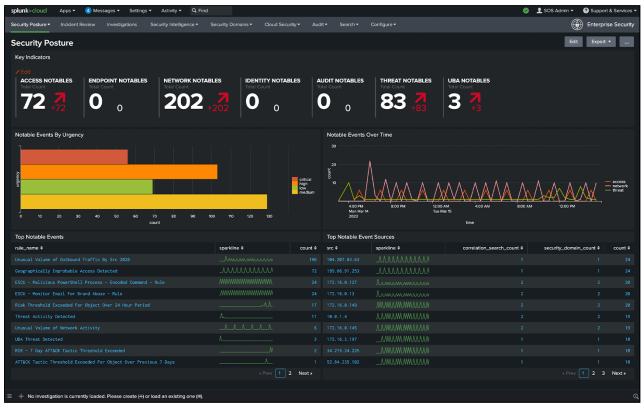

Fig. 5: The dashboard of Splunk – source: [121]

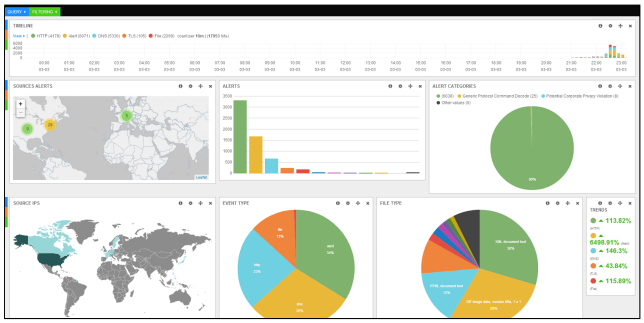

Fig. 6: The dashboard of Suricata – source: [86]

- "No amount of source-level verification or scrutiny will protect you from using untrusted code." [125].
- "A security audit subsystem is responsible for capturing, analyzing, reporting, archiving, and retrieving records of events and conditions within a computing solution. [...] can include [...] intrusion detection components. Security requirements for an audit subsystem would include: *trusted transfer of audit data*; *protection of security audit data*; analysis of security audit data—including review, anomaly detection, violation analysis, and attack analysis using simple or complex heuristics; alarms for loss thresholds." Additionally, the security audit may be used by the "solution integrity subsystem" which should focus on (among others) "*functional isolation using domain separation* or a reference monitor" [134].
- "The reference validation mechanism must be Tamper-proof. Without this property, an attacker can undermine the mechanism itself and hence violate the security policy." [17]
- "As a rule, if the software you are running on top of, whether it be an operating system, a piece of middleware, or something else, is insecure, what's above it is going to also be insecure" [28, 110].

Altogether, the aforementioned statements support the thesis that, to fulfill its security-related functions, all components of a security-focused system must not have been compromised—which is the crux of Assertion 1.